

UNIVERSITY OF BIRMINGHAM

School of Computer Science

Secure Software and Hardware Systems

Main Summer Examinations 2021

Secure Software and Hardware Systems

Exam paper

Question 1

- (a) Modern operating systems have mechanisms to limit the effect of buffer overflows in all binaries running on the system, without requiring recompilation or other changes. Explain the operation of two such mechanisms. As well as describing each independently, you should explain how your two mechanisms work together to provide additional protection.

Note: a “stack canary” is **not** effective for all binaries.

[10 marks]

- (b) Assume a system which uses all of the protection mechanisms discussed in this question (your answer to part (a) of this question). Consider this piece of code, which is supposed to perform administrative tasks if the correct administrative password is supplied as an argument

```
1  /* function which returns 1 if the admin password is correct,
2     0 if it is incorrect */
3  int check_password (char *);
4
5  void admin_command (char *password) {
6      char command[50];
7      int privileged = 0;
8
9      /* check the password is correct */
10     privileged = check_password (password);
11     /* read a command */
12     gets (command);
13
14     /* execute this if privileged is true */
15     if (privileged != 0) {
16         /* compare first six characters of the supplied command */
17         if (strncmp (command, "reboot", 6) == 0) {
18             system ("reboot");
19         }
20         /* other privileged commands could similarly go here */
21     }
22 }
```

Explain why this piece of code might under some circumstances permit a user who does not know the password to reboot the machine. Be clear about the assumptions you are making which allow your attack to work. **[4 marks]**

- (c) This example relies on obviously careless programming and the use of deliberately weak functions. However, similar attacks can hide in superficially well-written code. Suggest two techniques you might use when inspecting and testing a codebase which might detect such hidden problems, and explain how they work. **[6 marks]**

Question 2

As discussed in the lectures, the `xtime()` function in AES can easily lead to timing side-channel leakage. Per the standard, `xtime()` is implemented as follows:

```
1 uint8_t xtime(const uint8_t value) {  
2     uint8_t result = value << 1;  
3     if((value >> 7) & 1)  
4         result ^= 0x1B;  
5     return result;  
6 }
```

To prevent timing attacks, someone implements the function as `xtime_secure()` as follows:

```
1 uint8_t xtime_secure(const uint8_t value) {  
2     uint8_t result = value << 1;  
3     result = result ^ (((value >> 7) & 1) * 0x1B);  
4     return result;  
5 }
```

- (a) Briefly explain why the two implementations are functionally equivalent. **[3 marks]**
- (b) Assume that the processor executing `xtime_secure()` has a hardware multiplier to compute $a * b$. The multiplier returns zero after one clock cycle if either a or b is zero. Otherwise, the computation takes four clock cycles. Can this behaviour be used to mount a timing attack on the whole AES implementation? Explain your answer. **[4 marks]**
- (c) Now, assume that the multiplication always takes the same time, independent of the values of a and b . Will this (i) prevent timing attacks and (ii) prevent power side-channel attacks? Explain your answers. **[4 marks]**
- (d) In another part of the AES implementation, the following computation is done:
 $b = S[x \wedge k]$. Under which circumstances can this introduce timing leakage into the AES implementation? **[2 marks]**
- (e) The AES implementation is the only code running on a smartcard. You can only externally interact with the smartcard processor by giving it plaintext and receiving the corresponding ciphertext. Someone claims that because the smartcard processor has an L1 data cache, the AES implementation is vulnerable to a flush-and-reload attack. Is this correct? Explain your answer. **[3 marks]**
- (f) The smartcard also provides a signature function, which computes an RSA signature on a given value after applying some random padding. The RSA signature uses the

Chinese Remainder Theorem (CRT) optimisation. By means of fault injection, it is possible to set any chosen 16-byte block in the smartcard memory to all-zero. You however do not know the memory addresses of the variables used in the RSA-CRT computation. Outline how you can use this fault model to factor the RSA modulus.

[4 marks]